

Feature extraction



Contents

1 특징 추출

1.1 에지

1.2 미분을 이용한 에지 검출

1.3 다양한 미분관련 필터

1.4 쇼벨필터를 이용

1.5 샤프 필터

1.6 쇼벨 필터 적용

1.7 그래디언트

1.8 영상에서 그래디언트의 의미

1.9 벡터의 크기 계산

1.10 방향 계산

1.11 쇼벨 에지 추출 코드

1.12 결과

1.13 캐니 에지 검출 1

1.14 캐니 에지 검출 2

1.15 캐니 에지 검출 3

1.16 캐니 에지 검출 4

1.17 캐니 에지 검출 함수

1.18 캐니 에지 결과

1.19 허프 변환 직선 검출

1.20 허프변환을 이용한 직선 검출 함수

1.21 확률적 허프 변환에 의한 직선 검출

1.22 허프 직선 검출 예제 코드

1.23 결과

1.24 허프 원 검출 함수

1.25 허프 원 검출 - 코드

1.26 Result

1.27 동전 카운터

1.28 먼저 동전 검출

1.29 색상 구별로 동전의 값 찾기

1.30 HSV 색상

1.31 코드

1.32 결과

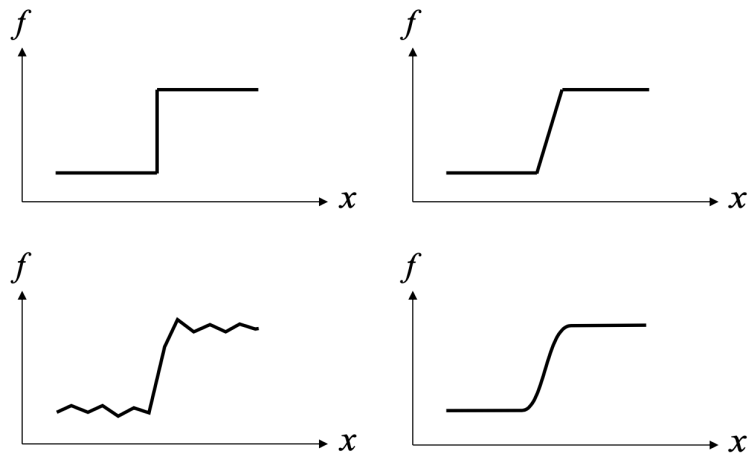
Feature extraction

Exported from Confluence · <https://pinkwink.atlassian.net>

특징 추출

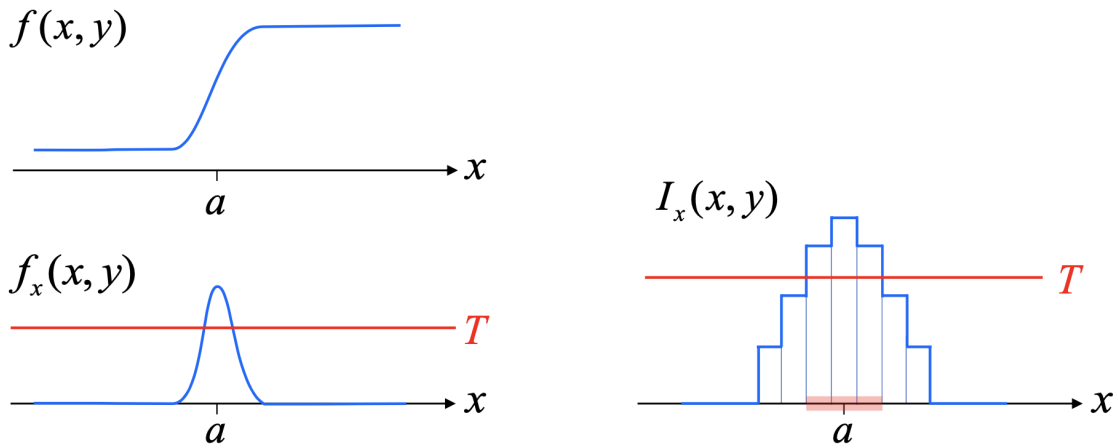
에지

- 영상에서 픽셀의 밝기 값이 급격하게 변하는 부분
- 일반적으로 배경과 객체, 또는 객체와 객체의 경계



미분을 이용한 에지 검출

- 영상을 (x, y) 변수의 함수로 간주했을 때, 이 함수의 1차 미분(1st derivative) 값이 크게 나타나는 부분을 검출



다양한 미분관련 필터

가로 방향:

-1	0	1
-1	0	1
-1	0	1

-1	0	1
-2	0	2
-1	0	1

-3	0	3
-10	0	10
-3	0	3

세로 방향:

-1	-1	-1
0	0	0
1	1	1

-1	-2	-1
0	0	0
1	2	1

-3	-10	-3
0	0	0
3	10	3

Prewitt

Sobel

Scharr

쇼벨필터를 이용

```
cv2.Sobel(src, ddepth, dx, dy, dst=None, ksize=None, scale=None,
          delta=None, borderType=None) -> dst
```

- src: 입력 영상
- ddepth: 출력 영상 데이터 타입. -1이면 입력 영상과 같은 데이터 타입을 사용.
- dx: x 방향 미분 차수.
- dy: y 방향 미분 차수.
- dst: 출력 영상(행렬)
- ksize: 커널 크기. 기본값은 3.
- scale: 연산 결과에 추가적으로 곱할 값. 기본값은 1.
- delta: 연산 결과에 추가적으로 더할 값. 기본값은 0.
- borderType: 가장자리 픽셀 확장 방식. 기본값은 cv2.BORDER_DEFAULT.

대부분 dx=1, dy=0, ksize=3 또는
dx=0, dy=1, ksize=3 으로 지정.

샤르 필터

```
cv2.Scharr(src, ddepth, dx, dy, dst=None, scale=None, delta=None,
            borderType=None) -> dst
```

- src: 입력 영상
- ddepth: 출력 영상 데이터 타입. -1이면 입력 영상과 같은 데이터 타입을 사용.
- dx: x 방향 미분 차수.
- dy: y 방향 미분 차수.
- dst: 출력 영상(행렬)
- scale: 연산 결과에 추가적으로 곱할 값. 기본값은 1.
- delta: 연산 결과에 추가적으로 더할 값. 기본값은 0.
- borderType: 가장자리 픽셀 확장 방식. 기본값은 cv2.BORDER_DEFAULT.

쇼벨 필터 적용

```

import sys
import numpy as np
import cv2

src = cv2.imread("./data/lenna.bmp", cv2.IMREAD_GRAYSCALE)

if src is None:
    print("Image load failed!")
    sys.exit()

dx = cv2.Sobel(src, -1, 1, 0, delta=128)
dy = cv2.Sobel(src, -1, 0, 1, delta=128)

cv2.imshow("src", src)
cv2.imshow("dx", dx)
cv2.imshow("dy", dy)
cv2.waitKey()

cv2.destroyAllWindows()

```

5.8s Python

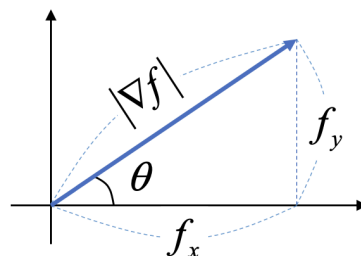
그레디언트

- 함수 $f(x, y)$ 를 x 축과 y 축으로 각각 편미분(partial derivative)하여 벡터 형태로 표현한 것

$$\nabla f = \begin{bmatrix} f_x \\ f_y \end{bmatrix} = f_x \mathbf{i} + f_y \mathbf{j}$$

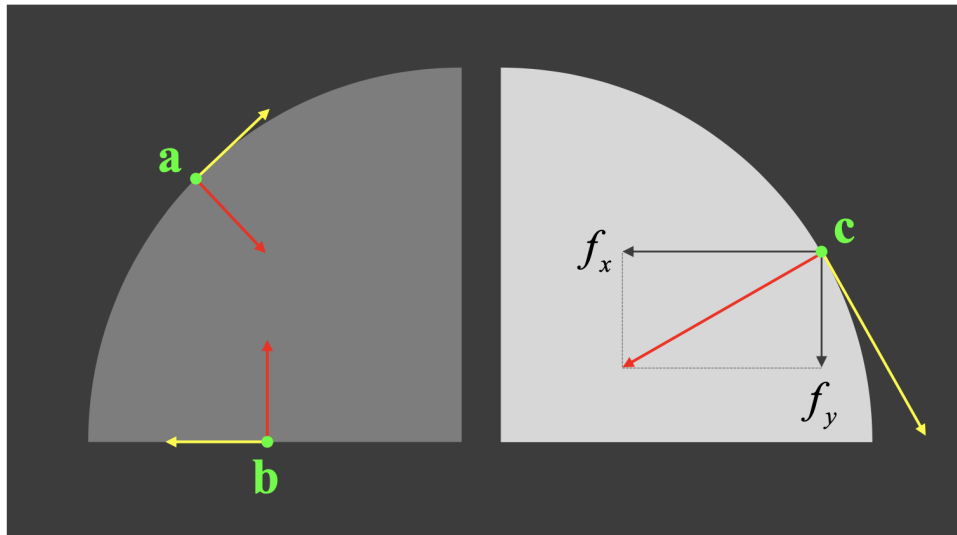
- 그레디언트 크기: $|\nabla f| = \sqrt{f_x^2 + f_y^2}$

- 그레디언트 방향: $\theta = \tan^{-1} \left(\frac{f_y}{f_x} \right)$



영상에서 그레디언트의 의미

- 그래디언트 크기: 픽셀 값의 차이 정도, 변화량
- 그래디언트 방향: 픽셀 값이 가장 급격하게 증가하는 방향



벡터의 크기 계산

```
cv2.magnitude(x, y, magnitude=None) -> magnitude
```

- x: 2D 벡터의 x 좌표 행렬. 실수형.
- y: 2D 벡터의 y 좌표 행렬. x와 같은 크기. 실수형.
- magnitude: 2D 벡터의 크기 행렬. x와 같은 크기, 같은 타입.

$$\text{magnitude}(I) = \sqrt{x(I)^2 + y(I)^2}$$

방향 계산

```
cv2.phase(x, y, angle=None, angleInDegrees=None) -> angle
```

- x: 2D 벡터의 x 좌표 행렬. 실수형.
- y: 2D 벡터의 y 좌표 행렬. x와 같은 크기. 실수형.
- angle: 2D 벡터의 크기 행렬. x와 같은 크기, 같은 타입.

$$\text{angle}(I) = \text{atan2}(y(I), x(I))$$

만약 $x(I)=y(I)=0$ 이면 angle은 0으로 설정됨.
- angleInDegrees: True이면 각도 단위, False이면 라디언 단위.

쇼벨 에지 추출 코드

```
import sys
import numpy as np
import cv2

src = cv2.imread('./data/lenna.bmp', cv2.IMREAD_GRAYSCALE)

if src is None:
    print('Image load failed!')
    sys.exit()

dx = cv2.Sobel(src, cv2.CV_32F, 1, 0)
dy = cv2.Sobel(src, cv2.CV_32F, 0, 1)

mag = cv2.magnitude(dx, dy)
mag = np.clip(mag, 0, 255).astype(np.uint8)

dst = np.zeros(src.shape[:2], np.uint8)
_, dst = cv2.threshold(mag, 120, 255, cv2.THRESH_BINARY)

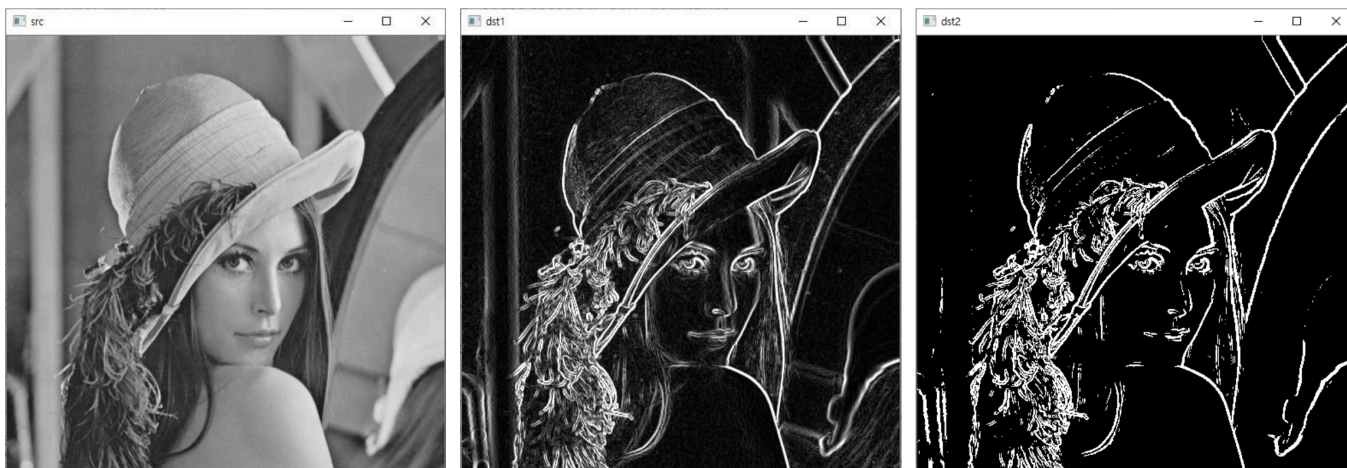
cv2.imshow('src', src)
cv2.imshow('mag', mag)
cv2.imshow('dst', dst)
cv2.waitKey()

cv2.destroyAllWindows()
```

✓ 5.1s

Python

결과

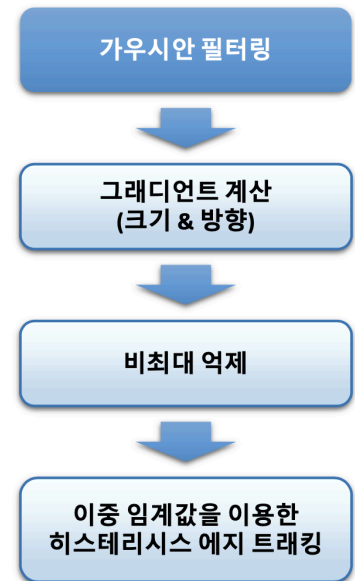


캐니 에지 검출 1

■ 캐니 에지 검출 1단계

- 가우시안 필터링
 - (Optional) 잡음 제거 목적

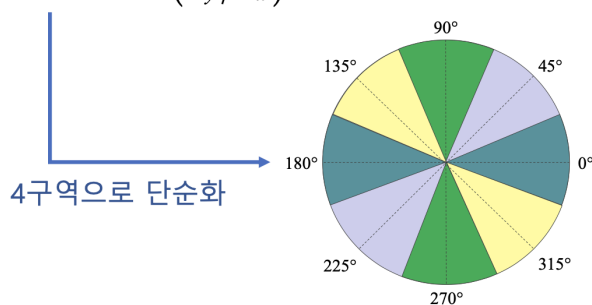
$$G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$



캐니 에지 검출 2

■ 캐니 에지 검출 2단계

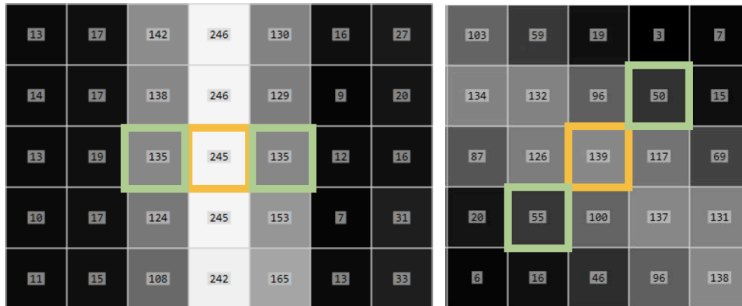
- 그라디언트 계산
 - 주로 소벨 마스크를 사용
 - 크기: $\|f\| = \sqrt{f_x^2 + f_y^2}$
 - 방향: $\theta = \tan^{-1}(f_y/f_x)$



캐니 에지 검출 3

■ 캐니 에지 검출 3단계

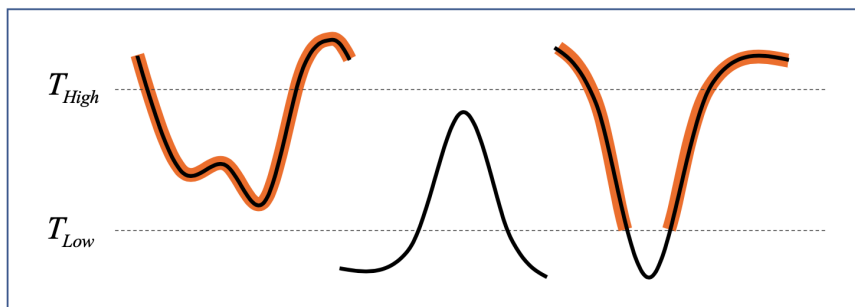
- 비최대 억제 (Non-maximum suppression)
 - 하나의 에지가 여러 개의 픽셀로 표현되는 현상을 없애기 위하여 그래디언트 크기가 국지적 최대 (local maximum)인 픽셀만을 에지 픽셀로 설정
 - 그래디언트 방향에 위치한 두 개의 픽셀을 조사하여 국지적 최대를 검사



캐니 에지 검출 4

■ 캐니 에지 검출 4단계

- 히스테리시스 에지 트래킹 (Hysteresis edge tracking)
 - 두 개의 임계값을 사용: T_{Low} , T_{High}
 - 강한 에지: $\|f\| \geq T_{High} \rightarrow$ 최종 에지로 선정
 - 약한 에지: $T_{Low} \leq \|f\| < T_{High}$
 $\rightarrow$ 강한 에지와 연결되어 있는 픽셀만 최종 에지로 선정



캐니 에지 검출 함수

```
cv2.Canny(image, threshold1, threshold2, edges=None, apertureSize=None,
          L2gradient=None) -> edges
```

- image: 입력 영상
- threshold1: 하단 임계값
- threshold2: 상단 임계값
- edges: 에지 영상
- apertureSize: 소벨 연산을 위한 커널 크기. 기본값은 3.
- L2gradient: True이면 L2 norm 사용, False이면 L1 norm 사용. 기본값은 False.

} threshold1:threshold2 = 1:2 또는 1:3

$$L_2 \text{ norm} = \sqrt{(dI/dx)^2 + (dI/dy)^2}, L_1 \text{ norm} = |dI/dx| + |dI/dy|$$

캐니 에지 결과

```
import sys
import numpy as np
import cv2

src = cv2.imread("./data/building.jpg", cv2.IMREAD_GRAYSCALE)

if src is None:
    print("Image load failed!")
    sys.exit()

dst = cv2.Canny(src, 50, 150)

cv2.imshow("src", src)
cv2.imshow("dst", dst)
cv2.waitKey()

cv2.destroyAllWindows()
```

✓ 1.6s

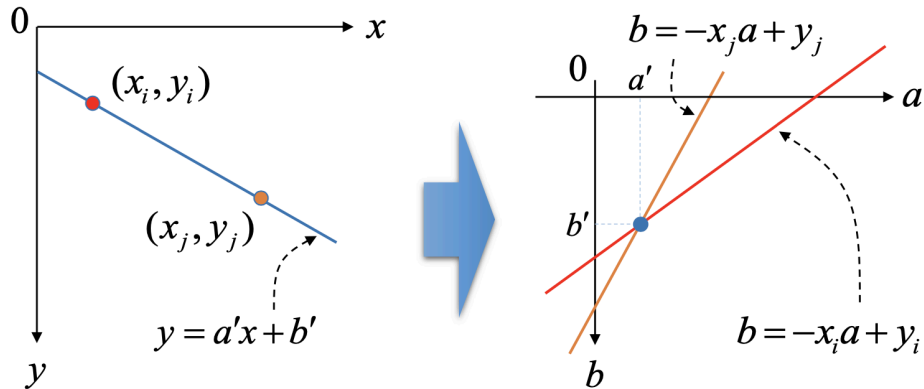
Python



허프 변환 직선 검출

- 2차원 영상 좌표에서의 직선의 방정식을 파라미터(parameter) 공간으로 변환하여 직선을 찾는 알고리즘

$$y = ax + b \Leftrightarrow b = -xa + y$$



허프변환을 이용한 직선 검출 함수

```
cv2.HoughLines(image, rho, theta, threshold, lines=None, srn=None, stn=None,
               min_theta=None, max_theta=None) -> lines
```

- image: 입력 에지 영상
- rho: 축적 배열에서 rho 값의 간격. (e.g.) 1.0 → 1픽셀 간격.
- theta: 축적 배열에서 theta 값의 간격. (e.g.) $\text{np.pi} / 180 \rightarrow 1^\circ$ 간격.
- threshold: 축적 배열에서 직선으로 판단할 임계값
- lines: 직선 파라미터(rho, theta) 정보를 담고 있는 `numpy.ndarray`.
shape=(N, 1, 2). dtype=`numpy.float32`.
- srn, stn: 멀티 스케일 허프 변환에서 rho 해상도, theta 해상도를 나누는 값.
기본값은 0이고, 이 경우 일반 허프 변환 수행.
- min_theta, max_theta: 검출할 선분의 최소, 최대 theta 값

확률적 허프 변환에 의한 직선 검출

```
cv2.HoughLinesP(image, rho, theta, threshold, lines=None,
                 minLineLength=None, maxLineGap=None) -> lines
```

- image: 입력 에지 영상
- rho: 축적 배열에서 rho 값의 간격. (e.g.) 1.0 → 1픽셀 간격.
- theta: 축적 배열에서 theta 값의 간격. (e.g.) $\text{np.pi} / 180 \rightarrow 1^\circ$ 간격.
- threshold: 축적 배열에서 직선으로 판단할 임계값
- lines: 선분의 시작과 끝 좌표(x1, y1, x2, y2) 정보를 담고 있는 `numpy.ndarray`.
shape=(N, 1, 4). dtype=`numpy.int32`.
- minLineLength: 검출할 선분의 최소 길이
- maxLineGap: 직선으로 간주할 최대 에지 점 간격

허프 직선 검출 예제 코드

```
import sys
import numpy as np
import cv2

src = cv2.imread("building.jpg", cv2.IMREAD_GRAYSCALE)

if src is None:
    print("Image load failed!")
    sys.exit()

edges = cv2.Canny(src, 50, 150)

lines = cv2.HoughLinesP(
    image = edges,
    rho = 1,
    theta = np.pi / 180.0,
    threshold = 160,
    minLineLength=50,
    maxLineGap=5)

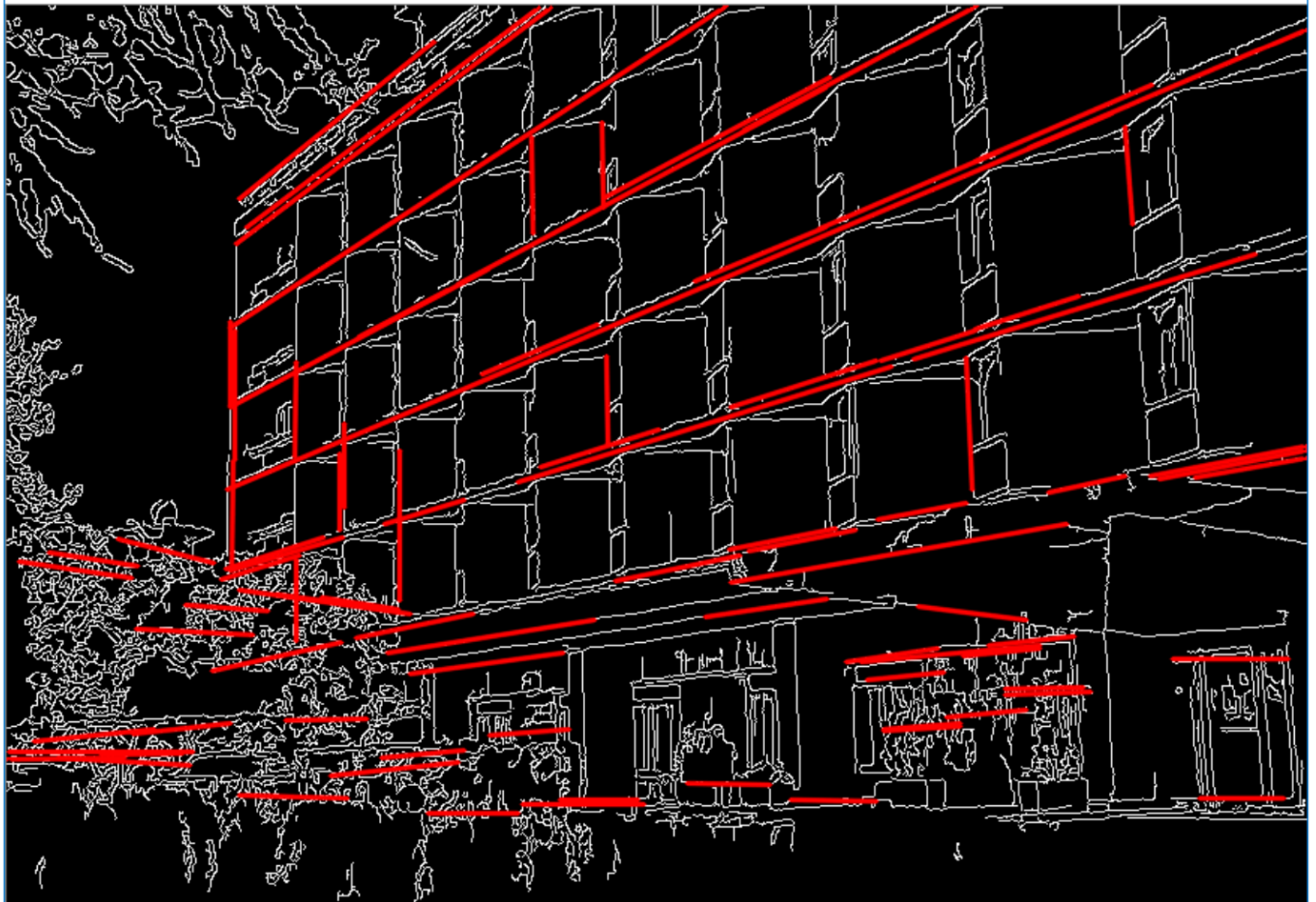
dst = cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)

if lines is not None:
    for i in range(lines.shape[0]):
        pt1 = (lines[i][0][0], lines[i][0][1]) # 시작점 좌표
        pt2 = (lines[i][0][2], lines[i][0][3]) # 끝점 좌표
        cv2.line(dst, pt1, pt2, (0, 0, 255), 2, cv2.LINE_AA)

cv2.imshow("src", src)
cv2.imshow("dst", dst)
cv2.waitKey()
cv2.destroyAllWindows()
```

Python

결과



허프 원 검출 함수

```
cv2.HoughCircles(image, method, dp, minDist, circles=None, param1=None,
                 param2=None, minRadius=None, maxRadius=None) -> circles
```

- image: 입력 영상. (에지 영상이 아닌 일반 영상)
- method: OpenCV 4.2 이하에서는 `cv2.HOUGH_GRADIENT`만 지정 가능
- dp: 입력 영상과 축적 배열의 크기 비율. 1이면 동일 크기. 2이면 축적 배열의 가로, 세로 크기가 입력 영상의 반.
- minDist: 검출된 원 중심점들의 최소 거리
- circles: (cx, cy, r) 정보를 담은 `numpy.ndarray`. shape=(1, N, 3), dtype=np.float32.
- param1: Canny 에지 검출기의 높은 임계값
- param2: 축적 배열에서 원 검출을 위한 임계값
- minRadius, maxRadius: 검출할 원의 최소, 최대 반지름

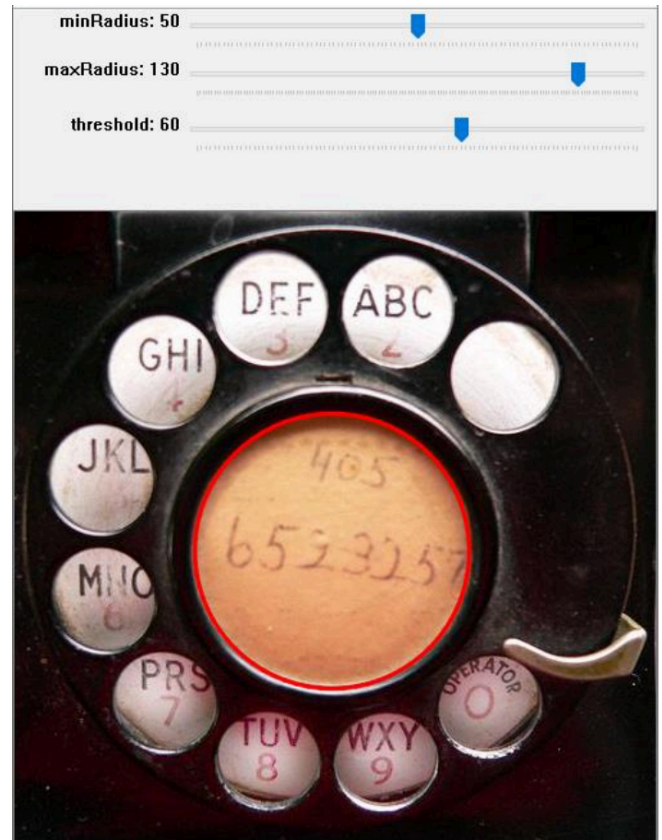
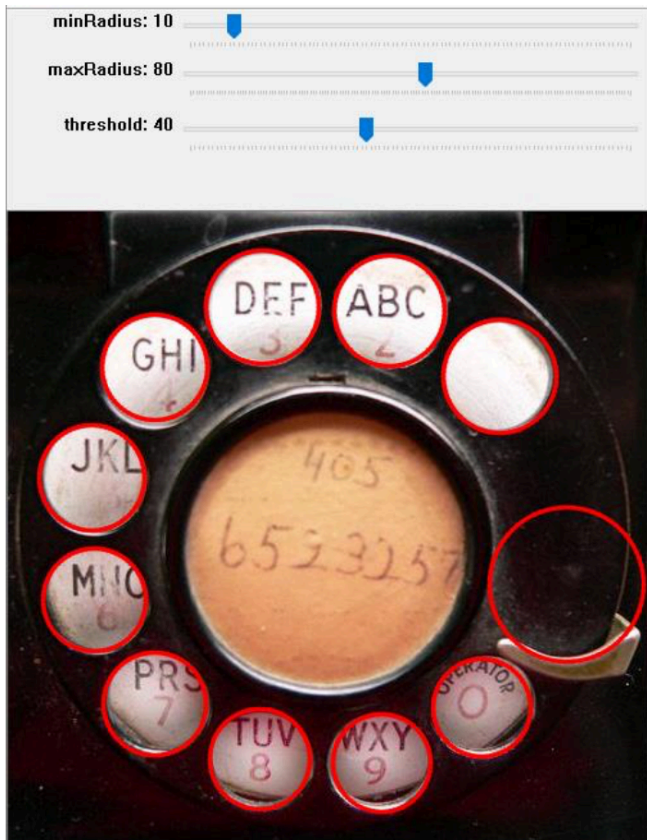
허프 원 검출 - 코드

```

1 import sys
2 import cv2
3
4 src = cv2.imread("./data/dial.jpg")
5
6 if src is None:
7     print("Image open failed!")
8     sys.exit()
9
10 gray = cv2.cvtColor(src, cv2.COLOR_BGR2GRAY)
11 blr = cv2.GaussianBlur(gray, (0, 0), 1.0)
12
13
14 def on_trackbar(pos):
15     rmin = cv2.getTrackbarPos("minRadius", "img")
16     rmax = cv2.getTrackbarPos("maxRadius", "img")
17     th = cv2.getTrackbarPos("threshold", "img")
18
19     circles = cv2.HoughCircles(
20         image=blr,
21         method=cv2.HOUGH_GRADIENT,
22         dp=1,
23         minDist=50,
24         param1=120,
25         param2=th,
26         minRadius=rmin,
27         maxRadius=rmax,
28     )
29
30     dst = src.copy()
31     if circles is not None:
32         for i in range(circles.shape[1]):
33             cx, cy, radius = circles[0][i]
34             cv2.circle(
35                 dst,
36                 (int(cx), int(cy)),
37                 int(radius),
38                 (0, 0, 255),
39                 2,
40                 cv2.LINE_AA
41             )
42
43     cv2.imshow("img", dst)
44
45 cv2.imshow("img", src)
46 cv2.createTrackbar("minRadius", "img", 1, 100, on_trackbar)
47 cv2.createTrackbar("maxRadius", "img", 1, 150, on_trackbar)
48 cv2.createTrackbar("threshold", "img", 1, 100, on_trackbar)
49 cv2.setTrackbarPos("minRadius", "img", 10)
50 cv2.setTrackbarPos("maxRadius", "img", 80)
51 cv2.setTrackbarPos("threshold", "img", 40)
52 cv2.waitKey()
53
54 cv2.destroyAllWindows()

```

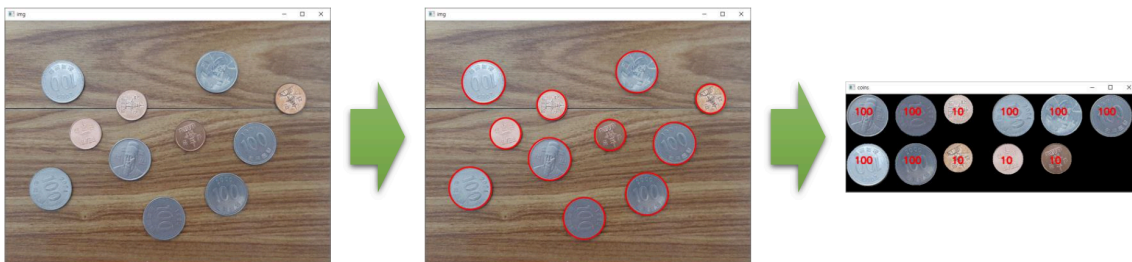
Result



동전 카운터

■ 동전 카운터

- 영상의 동전을 검출하여 금액이 얼마인지를 자동으로 계산하는 프로그램
- 편의상 동전은 100원짜리와 10원짜리만 있다고 가정



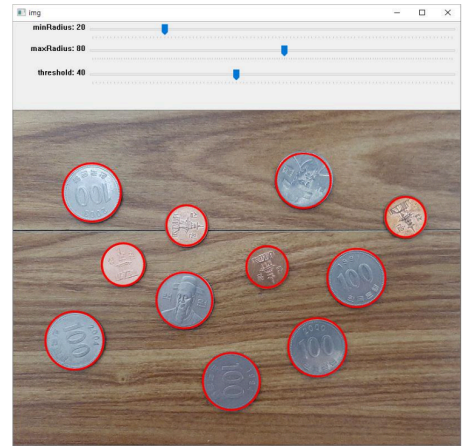
■ 구현할 기능

- 동전 검출하기 → 허프 원 검출
- 동전 구분하기 → 색상 정보 이용

먼저 동전 검출

■ 동전 검출하기

- 동그란 객체는 동전만 있다고 가정
→ cv2.HoughCircles() 함수 사용
- 영상 크기: 800x600 (px)
- 동전 크기
 - 100원: 약 100x100 (px)
 - 10원: 약 80x80 (px)



```
src = cv2.imread('coins2.jpg')
gray = cv2.cvtColor(src, cv2.COLOR_BGR2GRAY)
blr = cv2.GaussianBlur(gray, (0, 0), 1)

circles = cv2.HoughCircles(blr, cv2.HOUGH_GRADIENT, 1, 50,
                             param1=150, param2=40, minRadius=20, maxRadius=80)
```

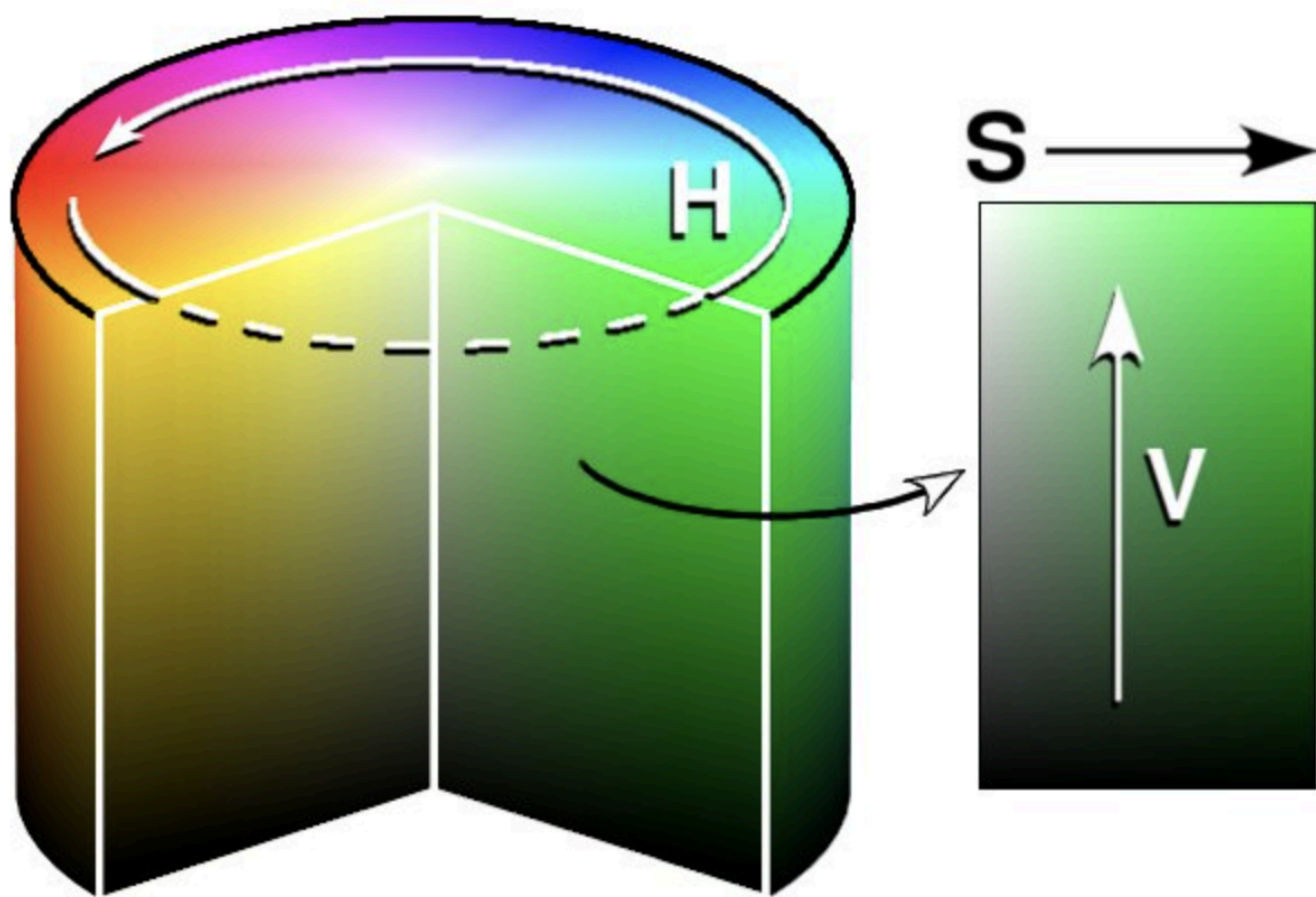
색상 구별로 동전의 값 찾기

- 동전 영역 부분 영상 추출 → HSV 색 공간으로 변환
- 동전 영역에 대해서만 Hue 색 성분 분포 분석



- 동전 영역 픽셀에 대해 Hue 값을 +40만큼 시프트하고, Hue 평균을 분석
 - Hue 평균이 90보다 작으면 10원
 - Hue 평균이 90보다 크면 100원

HSV 색상



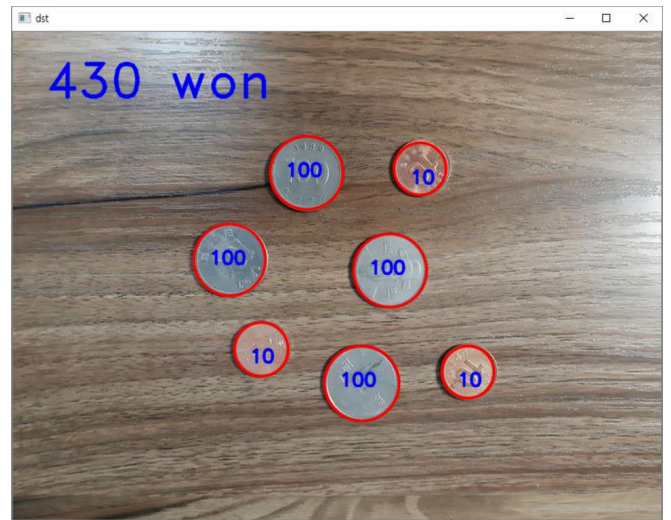
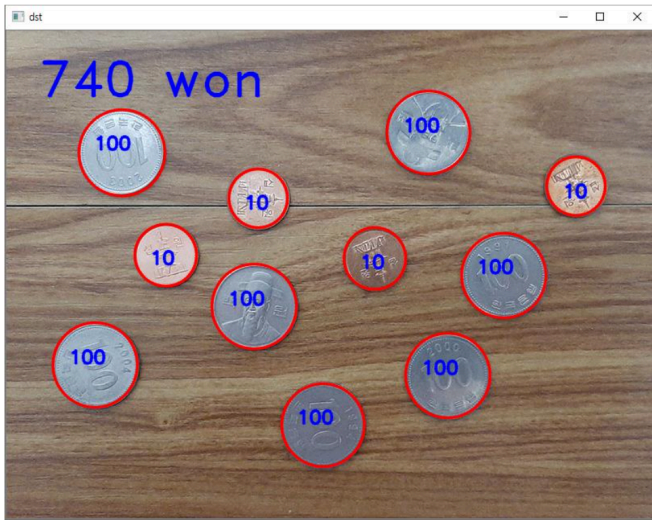
코드

```

1 import sys
2 import numpy as np
3 import cv2
4
5 # 입력 이미지 불러오기
6 src = cv2.imread('./data/coins1.jpg')
7
8 if src is None:
9     print('Image open failed!')
10    sys.exit()
11
12 gray = cv2.cvtColor(src, cv2.COLOR_BGR2GRAY)
13 blr = cv2.GaussianBlur(gray, (0, 0), 1)
14
15 # 허프 변환 원 검출
16 circles = cv2.HoughCircles(blr, cv2.HOUGH_GRADIENT, 1, 50,
17                             param1=150, param2=40, minRadius=20, maxRadius=80)
18
19 # 원 검출 결과 및 동전 금액 출력
20 sum_of_money = 0
21 dst = src.copy()
22 if circles is not None:
23     for i in range(circles.shape[1]):
24         cx, cy, radius = circles[0][i]
25         cx, cy, radius = list(map(int, [cx, cy, radius]))
26         cv2.circle(dst, (cx, cy), radius, (0, 0, 255), 2, cv2.LINE_AA)
27
28         # 동전 영역 부분 영상 추출
29         x1 = int(cx - radius)
30         y1 = int(cy - radius)
31         x2 = int(cx + radius)
32         y2 = int(cy + radius)
33         radius = int(radius)
34
35         crop = dst[y1:y2, x1:x2, :]
36         ch, cw = crop.shape[:2]
37
38         # 동전 영역에 대한 ROI 마스크 영상 생성
39         mask = np.zeros((ch, cw), np.uint8)
40         cv2.circle(mask, (int(cw//2), int(ch//2)), radius, 255, -1)
41
42         # 동전 영역 Hue 색 성분을 +40 시프트하고, Hue 평균을 계산
43         hsv = cv2.cvtColor(crop, cv2.COLOR_BGR2HSV)
44         hue, _, _ = cv2.split(hsv)
45         hue_shift = (hue + 40) % 180
46         mean_of_hue = cv2.mean(hue_shift, mask)[0]
47
48         # Hue 평균이 90보다 작으면 10원, 90보다 크면 100원으로 간주
49         won = 100
50         if mean_of_hue < 53:
51             won = 10
52
53         sum_of_money += won
54
55         cv2.putText(crop, str(won), (20, 50), cv2.FONT_HERSHEY_SIMPLEX,
56                     0.75, (255, 0, 0), 2, cv2.LINE_AA)
57
58 cv2.putText(dst, str(sum_of_money) + ' won', (40, 80),
59             cv2.FONT_HERSHEY_DUPLEX, 2, (255, 0, 0), 2, cv2.LINE_AA)
60
61 cv2.imshow('src', src)
62 cv2.imshow('dst', dst)
63 cv2.waitKey()
64
65 cv2.destroyAllWindows()

```

결과





PinkLAB YouTube Channel

PinkLAB Studio — Robotics, AI, ROS2 and more!

 **Subscribe on YouTube**

https://www.youtube.com/@pinklab_studio